Specification — Software Design

Document ID	SDS-SPEC-001
Revision	dfa6eda
Date	2026-11-10
Status	Controlled in git

Contents

1 Purpose 3
2 Design Outputs 3
3 Dependencies 5
4 Out of scope 5

Id: `SDS-SPEC-001` · Context: `specification`

1 Purpose

The core bounded context: the spec the record is held to, read and never written. RDM's contract with a spec is Gherkin: a requirement is a scenario tagged with an id, and a user need is, by default, a Feature. Where the feature files are, what an id looks like and where user needs come from are the DHF's configuration (`spec:` in `config.yml`), or an adapter for one layout (Löyly's). Every other context conforms to its ids. It also reads the validation records of the user needs. Onboarding (`rdm init`, `rdm adopt`), the commands that lay a project down, is this context's application layer.

2 Design Outputs

Component	Responsibility	Meets
Gherkin reader	Parses feature files with Cucumber's Gherkin parser: each scenario (or outline) tagged with an id (any id-shaped tag, or the configured pattern) is a requirement, with its Feature, Rule, title, source line and tags; the last id tag of a scenario is its id; each untagged scenario is listed; a file that does not parse is a problem, never skipped.	item-01-001
Spec configuration	Finds the spec the record is held to (`spec` in the DHF's `config.yml`: feature globs, exclusions, id pattern, needs source, layout; else `.pi/plan`, `spec/` or `features/` from the repository root) and its user needs: by default each Feature, addressed by the scenarios under it.	item-01-001, item-01-005
Löyly adapter	An optional layout and needs source: Löyly's plan of slices (planned and backlog skipped, `drafts.feature` never read) and its user needs, the event-model screens with their Allium surfaces, actors, rules and guarantees and the event-storm events behind them.	item-01-001, item-01-005

Component	Responsibility	Meets
Project scaffold	`rdm init`: copies the project templates into a new directory; refuses a directory that exists (exit 2), and on success names what it laid down and the next steps.	item-01-002
Project templates	What `rdm init` lays down: the document templates (the traceability matrix among them), the build Makefile (a Word document is rebuilt when its reference document changes), the render configuration, the container build (installing the RDM release from git when no wheel is given), and the Pandoc and Typst configuration. Templates use only local images.	item-01-002
Adoption	`rdm adopt`: creates each missing file of the adoption templates and skips, never overwrites, one that exists (a symbolic link exists, dangling or not); when a `.gitignore` exists, names the lines of RDM's generated files to add to it; stamps the installed version as its release tag (a pre-release such as 2.0.0a0 as 2.0.0-alpha, the tag the release and its image carry); reports what it skipped and prints the next steps.	item-01-003
Adoption templates	What `rdm adopt` lays down: the DHF skeleton (V&V plan, traceability matrix, render configuration, README), a CI gate workflow and a `.gitignore` of RDM's generated files (Allure results, verification data, the graph store, bytecode).	item-01-003
Validation records	Reads the validation records from the DHF's validation directory, each naming a user need by its id; only an approved disposition with a named reviewer counts. Gives the user needs without one.	item-01-004

- The Gherkin reader reads every feature file the spec configuration finds, or that the Löyly adapter's layout lists.
- The spec configuration takes its user needs from the Features, or hands the context directories to the Löyly adapter for its needs.
- Validation records read the user needs from the spec.
- Project scaffold and adoption copy their templates.

3 Dependencies

Layer 2 of the dependency rule. Depends on the shared kernel only (YAML, EDN for the adapter) and on Cucumber's Gherkin parser. Depended on by `test_evidence` (the pytest plugin reads the requirement ids), `release` (the release gate, the verification data), `publishing` (the DMR index, the verification report, the evidence bundle) and `graph` (the projection). The composition root wires its commands. It reads no results and no risk register.

4 Out of scope

- Writing or stamping the spec: whoever owns it does (ADR 0002).
- Usability validation by simulated users: the human validation record is the evidence.
- Planning: tasks and issues live outside the record.